

常州工学院

CHANGZHOU INSTITUTE OF TECHNOLOGY

课程大作业说明书

课程名： 大数据与云计算

题 目： 图书管理系统

二级学院： 计算机信息工程学院

专 业： 软件工程

班 级： 23 软件一

学号姓名： 23030327 许子祺

23030315 莫文涛

23030316 钱季清

23250123 王奕泽

指导教师： 李亦飞

2026 年 6 月

摘 要

本文围绕“移动应用开发技术”课程大作业，设计并实现了一套面向幼儿园图书流转场景的鸿蒙移动端图书管理系统。系统以 Flutter 前端为基础，通过 Flutter for OpenHarmony 生成 HarmonyOS 工程，并在 ArkTS 入口页中嵌入 Flutter 渲染页面，使原有移动端界面和业务流程可以在 DevEco Studio、HarmonyOS 模拟器以及后续真机环境中运行。应用支持教师和管理员两类角色，覆盖邮箱登录、图书检索、分类筛选、图书新增与编辑、库存维护、教师/学生借阅、归还登记、逾期查询、学生管理、个人中心和图书馆仪表盘等功能。

在后端设计上，系统将原先依赖的 Supabase 数据服务迁移为 PocketBase + Docker 本地化部署方案。迁移过程严格保留原 Supabase 中 books、categories、students、profiles 和 borrow_records 等列表字段及已有数据内容，仅在 PocketBase 身份认证约束需要时为 profiles 保存 source_id 映射字段。系统使用 Docker Compose 启动 PocketBase，并将 /pb_data 挂载到 Docker named volume kindergarten-pocketbase-data，避免 macOS Docker bind mount 对 SQLite 直接读写带来的偶发 I/O 抖动。同时，前端服务层加入登录状态收敛、轮询间隔控制和上一次成功数据缓存，解决了模拟器环境下偶发空列表、重复鉴权和图片路径解析不稳定等问题。

经过功能验证与构建验证，系统能够完成本地 PocketBase 数据访问、鸿蒙模拟器调试和 HAP 构建，形成了从数据迁移、后端部署、Flutter 适配到 HarmonyOS 应用打包的完整工程闭环。本文重点说明系统需求、总体架构、鸿蒙适配、数据库迁移、核心功能实现、测试结果与后续优化方向。

关键词：HarmonyOS；Flutter for OpenHarmony；PocketBase；Docker；图书管理；数据迁移

Abstract

This report presents the design and implementation of a HarmonyOS-oriented kindergarten library management system for a mobile application development course project. The system keeps the existing Flutter frontend as much as possible, generates an OpenHarmony project through Flutter for OpenHarmony, and embeds the Flutter page in the ArkTS entry ability. It supports two application roles, teacher and administrator, and provides email login, book search, category filtering, book creation and editing, inventory management, teacher/student borrowing, returning, overdue lookup, student management, personal center, and dashboard statistics.

On the backend side, the original Supabase-based data service is migrated to a local PocketBase service deployed with Docker. The migration keeps the original Supabase tables, fields, and data for books, categories, students, profiles, and borrow_records. Only the source_id mapping field is added to the PocketBase authentication collection to satisfy identity constraints. Docker Compose runs PocketBase locally, while /pb_data is stored in the Docker named volume kindergarten-pocketbase-data, reducing direct SQLite I/O pressure on the macOS Docker bind mount layer. In the Flutter service layer, auth refresh convergence, polling interval control, and last-successful-data caching are introduced to avoid occasional empty lists, repeated authentication refreshes, and unstable image URL resolution in the emulator environment.

The implemented system has been verified through local PocketBase access, HarmonyOS emulator debugging, and HAP build validation. It forms a complete engineering workflow covering database migration, backend deployment, Flutter adaptation, and HarmonyOS application packaging. This report focuses on the requirements, architecture, HarmonyOS adaptation, database migration, module implementation, testing, and future improvements.

Keywords: HarmonyOS; Flutter for OpenHarmony; PocketBase; Docker; Library Management; Data Migration

目 录

第 1 章 绪论	1
1.1 项目背景	1
1.2 开发目标	2
1.3 需求分析	2
1.3.1 业务场景分析	2
1.3.2 用户角色需求	3
1.3.3 功能需求	3
1.3.4 非功能需求	4
1.4 本文组织结构	4
第 2 章 系统总体设计	5
2.1 总体架构	5
2.2 技术栈设计	6
2.3 功能模块划分	6
2.4 数据模型设计	7
2.4.1 字段映射原则	8
2.4.2 数据一致性约束	8
2.5 借阅业务流程	9
2.6 接口调用时序	9
2.7 数据库迁移与性能优化设计	10
第 3 章 系统具体实现	13
3.1 鸿蒙工程适配实现	13
3.1.1 鸿蒙工程目录说明	13
3.1.2 权限与网络配置	14
3.2 后端连接与鉴权实现	14
3.2.1 服务层异常处理	14
3.2.2 角色路由实现	15
3.3 仪表盘模块	15
3.4 图书管理模块	16
3.5 分类管理模块	17
3.6 借阅与归还模块	17

3.7 学生管理模块	18
3.8 图片加载实现	20
3.9 界面适配与交互实现	20
第 4 章 测试与运行验证	21
4.1 测试环境	21
4.2 数据迁移验证	21
4.3 功能测试	22
4.4 界面适配测试	23
4.5 后端运行验证	24
4.6 构建验证	24
4.7 稳定性问题分析	25
4.8 问题定位复盘	26
第 5 章 总结与展望	27
5.1 工作总结	27
5.2 不足之处	27
5.3 未来展望	28
参考文献	29
附录 A 附录：核心命令与目录说明	30

第1章 绪论

1.1 项目背景

传统方式常依赖纸质表格或简单电子表格，能够满足少量图书的基本记录，但在图书数量增加、多人经办、班级流转频繁时，会出现库存不清、逾期记录难追踪、同一本书借出数量与实际剩余数量不一致等问题。对于教师而言，移动端直接完成图书查询、借阅登记和归还登记，比在电脑端维护表格更符合日常工作场景。

本课程大作业以“幼儿园图书管理系统”为主题，目标不是制作一个静态展示页面，而是完成一个能够在 HarmonyOS 环境下运行的真实移动端应用。项目原始前端基于 Flutter 开发，早期后端使用 Supabase。为了适配鸿蒙课程要求，同时降低第三方云服务和平台 SDK 的兼容风险，本项目保留 Flutter 前端主体，切换到 Flutter for OpenHarmony SDK 生成 ohos/ 工程，并将后端迁移为本地 Docker 部署的 PocketBase 服务^[1-4]。

图1.1展示了系统的应用入口。登录页采用邮箱和密码认证，登录后根据用户在 profiles 中的角色字段进入教师或管理员可访问的功能范围。



图 1.1 系统登录主界面

1.2 开发目标

本项目的总体目标是实现一个可运行、可构建、可验收的 HarmonyOS 图书管理应用。具体目标如下：

1. 完成 Flutter 前端向 OpenHarmony 工程的适配，生成可由 DevEco Studio 打开和构建的 ohos/ 工程^[5-6]。
2. 完成 Supabase 到 PocketBase 的数据迁移，保留原数据库中的列表字段和数据内容，不重新制造不存在的业务字段。
3. 使用 Docker Compose 提供本地 PocketBase 服务，支持稳定的数据持久化、封面静态资源访问和本地调试。
4. 实现图书、分类、学生、借阅记录、逾期记录、个人中心和仪表盘等完整业务模块。
5. 通过命令行完成 Flutter 测试、静态分析和 HarmonyOS HAP 构建验证，为后续真机验收提供基础。

1.3 需求分析

1.3.1 业务场景分析

幼儿园图书室的管理对象与高校图书馆或公共图书馆不同。首先，图书通常面向班级和教师流转，借阅对象既可能是学生，也可能是教师本人；其次，图书种类相对集中，更多关注绘本、幼儿读物、教学参考材料和班级活动用书；再次，管理人员往往不是专职图书管理员，而是由教师兼任。因此系统设计不能只追求复杂的编目能力，更需要降低操作门槛，使教师在移动设备上能够快速完成查询、借阅、归还和逾期处理。

在实际使用中，教师最常见的操作路径是：打开应用后查看图书总览，通过分类或关键词找到图书，再根据库存情况发起借阅；管理员更关注整体运行状态，例如当前有多少本书在借、哪些图书逾期、哪些学生或教师借阅次数较多。由此可见，本项目的重点并不是实现一个通用大型图书馆系统，而是围绕“小规模库存、多角色借阅、移动端快速操作、课程验收可复现”这四个条件完成一个完整闭环。

表1.1给出了系统面向的主要业务场景。需求分析阶段将这些场景拆解为页面、服务、数据表和测试用例，后续章节中的模块设计均围绕这些场景展开。

表 1.1 主要业务场景

场景	参与角色	系统响应
图书查询	教师、管理员	按书名、作者、位置和分类检索图书，展示库存与可借状态。
图书借阅	教师、管理员	校验可借库存，创建借阅记录，扣减可借数量。
图书归还	教师、管理员	写入归还日期，恢复可借库存，刷新个人与管理员记录。
逾期核查	管理员	根据应还日期和归还状态筛选逾期未还记录。
学生维护	管理员	管理学生姓名和班级，并关联学生借阅记录。
验收部署	开发者、验收人员	启动 Docker 后端，使用鸿蒙模拟器或真机访问同一数据服务。

1.3.2 用户角色需求

系统目前包含两类应用角色：普通教师和管理员。普通教师主要完成图书查询、借阅、归还和个人借阅记录查看；管理员在教师能力基础上，还需要查看全部借阅记录、逾期未还图书、图书分类与学生信息。角色信息存储在 PocketBase 的 `profiles auth collection` 中，由登录成功后的用户资料决定。

角色设计采用“最小权限”原则。普通教师不直接维护学生基础数据和全局借阅记录，避免误删或误改公共数据；管理员负责维护基础资料和全局查询，用于处理库存、逾期和分类等管理任务。由于课程验收需要快速切换角色，系统在 README 和测试章节中列出了教师与管理员账号，便于验证不同权限下的页面差异。

1.3.3 功能需求

系统核心功能包括以下几个方面：

- **登录与账号管理**：支持邮箱密码登录、注册、资料查看和角色识别。
- **图书管理**：支持按标题、作者、位置搜索图书，按分类筛选图书，新增和编辑图书信息，维护总库存与可借库存。
- **分类管理**：支持查看、添加、编辑和删除图书分类，并在图书筛选中直接使用分类数据。
- **借阅与归还**：支持教师或学生借阅图书，登记借阅日期、应还日期、数量和经办人，并在归还时恢复库存。

- **学生管理**：支持学生列表、班级信息、学生详情和个人借阅历史查询。
- **管理员查询**：支持全部借阅记录、未归还记录和逾期记录查询。
- **仪表盘统计**：展示图书总数、学生总数、当前在借、逾期未还、本月借阅次数、热门图书和借阅之星。

1.3.4 非功能需求

在课程设计场景下，系统还需要满足以下非功能要求：第一，鸿蒙端应能通过 DevEco Studio 和 CLI 构建，不依赖 Android 原生工程运行；第二，本地后端需要便于教师验收和复现，不依赖远程账号授权；第三，数据迁移需要可追溯，源库字段、记录数量和业务语义应保持一致；第四，模拟器和真机网络地址差异需要通过配置项解决；第五，界面应适配手机竖屏，避免按钮、库存标签和借阅数量等元素溢出。

除上述要求外，系统还需要具备一定的容错能力。移动端网络环境天然不稳定，鸿蒙模拟器访问宿主机服务时还存在地址转换问题，因此页面不能把一次请求失败直接解释为“业务数据为空”。本项目在服务层保留上一次成功读取的数据，并在登录态刷新、列表轮询和图片加载处减少重复请求，使用户在短时网络波动下仍能看到最近一次有效数据。

1.4 本文组织结构

本文后续章节安排如下：第2章介绍系统总体设计、技术架构、数据模型和迁移优化方案；第3章介绍鸿蒙适配与各业务模块的具体实现，并结合截图说明功能效果；第4章给出运行环境、测试过程、构建验证和稳定性分析；第5章总结项目成果并说明后续可优化方向。

第2章 系统总体设计

2.1 总体架构

系统采用“鸿蒙移动端 + Flutter 业务界面 + PocketBase 本地后端 + Docker 持久化”的轻量级前后端分离架构。移动端负责页面渲染、交互状态、表单校验和业务服务编排；PocketBase 负责身份认证、REST API、数据存储和静态资源；Docker Compose 负责本地后端启动、端口暴露和数据卷管理^[3-4,7]。

图2.1为使用 chat-gpt-image-2 生成的总体架构图。图中从左到右体现了 HarmonyOS 设备侧、Flutter/Dart 业务层、PocketBase REST API 和 Docker 持久化层之间的关系。

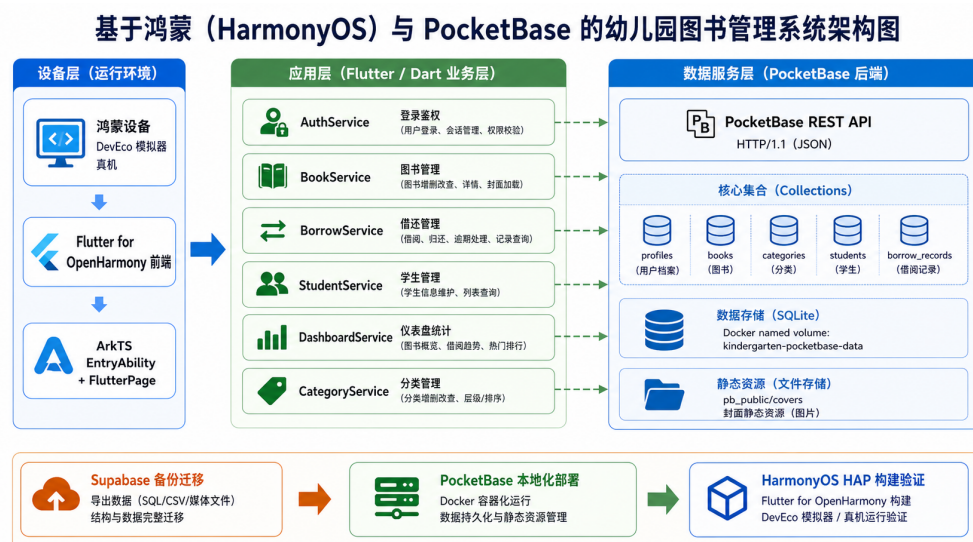


图 2.1 系统总体架构图

系统运行时，鸿蒙端通过 BackendConfig 获取 PocketBase 地址。模拟器访问宿主机 PocketBase 时使用 `http://10.0.2.2:8090`，真机验收时可通过 `--dart-define=POCKETBASE_URL=http://Mac 局域网 IP:8090` 指向同一后端服务。

从职责划分看，移动端并不直接操作数据库文件，而是通过 PocketBase SDK 发起 REST 请求；PocketBase 负责鉴权、数据校验和 SQLite 持久化；Docker 只负责运行环境隔离、端口映射和数据卷保存。这样的拆分使每一层都可以单独验证：前端可通过 Flutter 测试验证模型与服务逻辑，后端可通过 PocketBase 管理后台检查 collection 和记录，鸿蒙工程可通过 HAP 构建验证平台适配是否成功。

本项目没有采用“所有逻辑都放在页面组件中”的方式，而是在 Flutter 端保留

模型、服务和页面三层结构。页面层负责展示、输入和跳转；服务层负责调用 PocketBase、组合查询结果、处理异常和维护缓存；模型层负责类型转换和字段映射。该结构对于跨平台迁移尤其重要，因为 OpenHarmony 适配主要发生在外层工程与插件能力上，业务层保持 Dart 实现可以显著降低迁移风险。

2.2 技术栈设计

系统主要技术栈如表2.1所示。选型原则是尽量复用原 Flutter 前端，减少鸿蒙迁移时的页面重写成本，同时将高风险的云端 SDK 依赖替换为跨平台 HTTP/REST 访问方式。

表 2.1 系统技术栈

层次	技术选型与作用
移动端框架	Flutter / Dart，保留原页面、模型和业务服务组织方式。
鸿蒙适配	Flutter for OpenHarmony 3.22，生成 ohos/ 工程并接入 ArkTS EntryAbility ^[5-6] 。
开发工具	DevEco Studio 6.0.2、HarmonyOS SDK API 22、ohpm、hvisor ^[8] 。
后端服务	PocketBase，提供 auth collection、base collection、REST API 和静态文件服务 ^[3,7] 。
部署方式	Docker Compose，本地启动 PocketBase 并暴露 8090 端口 ^[4] 。
数据持久化	Docker named volume kindergarten-pocketbase-data 保存 /pb_data ^[9] 。
前端依赖	pocketbase、shared_preferences、cached_network_image、intl 等。

技术选型中最关键的取舍是数据库后端。Supabase 功能完整，但课程验收时需要远程服务、项目密钥和网络连通性；PocketBase 则可以通过单个容器提供认证、表结构、数据和静态资源，启动成本更低。由于 Flutter for OpenHarmony 对部分移动端插件和原生 SDK 的支持仍需要逐项验证，本项目选择用 HTTP/REST 边界隔离平台差异，把平台相关问题控制在网络地址、权限和构建配置范围内。

2.3 功能模块划分

系统按照幼儿园图书业务边界划分为登录鉴权、仪表盘、图书管理、分类管理、借阅归还、学生管理、管理员查询和个人中心等模块。图2.2展示了各模块之间的功能关系。

各模块之间通过服务层进行协作。例如，图书列表页面调用 BookService 读

幼儿园图书管理系统功能模块图

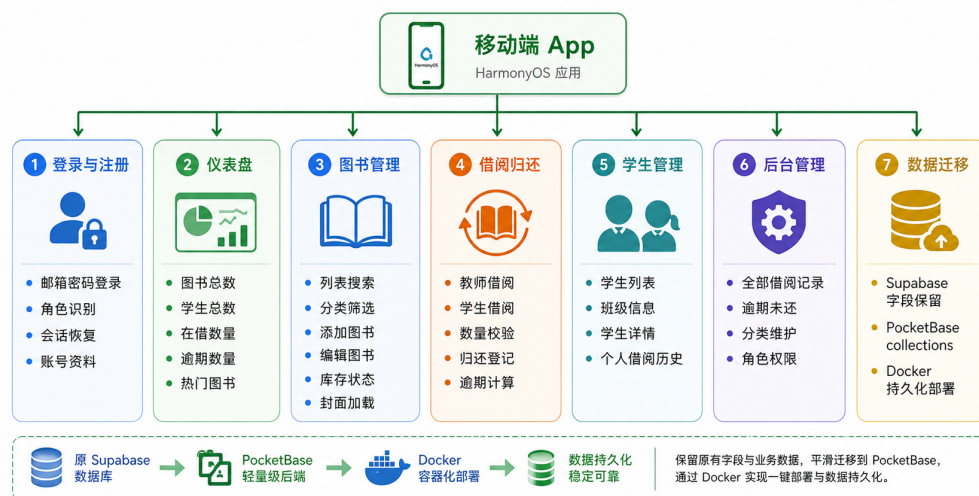


图 2.2 系统功能模块图

取图书与分类，借阅页面调用 `BorrowService` 先校验库存再创建借阅记录，仪表盘页面调用 `DashboardService` 汇总图书、学生和借阅记录。这样的分层使页面组件只关注界面状态，数据访问和业务规则集中在服务层。

表2.2进一步说明各模块的输入、输出和依赖关系。该表也可作为后续测试设计的依据，因为每个模块都对应一组可验证的用户行为。

表 2.2 功能模块职责划分

模块	主要输入	主要输出与依赖
登录鉴权	邮箱、密码、注册资料	登录 token、用户角色、个人资料，依赖 <code>profiles</code> 。
仪表盘	图书、学生、借阅记录	统计卡片、热门图书、本月借阅数据，依赖 <code>DashboardService</code> 。
图书管理	搜索词、分类、图书表单	图书列表、详情、编辑结果，依赖 <code>books</code> 和 <code>categories</code> 。
借阅归还	图书、借阅对象、数量、日期	借阅记录、库存变化、逾期状态，依赖 <code>BorrowService</code> 。
学生管理	学生姓名、班级	学生列表、学生详情、个人借阅历史。
管理员查询	搜索词、状态筛选	全部借阅、未归还、逾期记录。

2.4 数据模型设计

PocketBase 迁移后保留原 Supabase 的主要列表及字段，核心 collection 如表2.3所示。PocketBase 将 collection 分为 base collection 和 auth collection，本项目

正是利用 `auth collection` 承载用户登录资料^[3,7]。

表 2.3 PocketBase 核心 Collection 字段

Collection	字段说明
profiles	auth collection。业务字段为 <code>source_id</code> 、 <code>full_name</code> 、 <code>updated_at</code> 、 <code>role</code> ，系统字段提供邮箱和密码认证。
categories	<code>name</code> 、 <code>created_at</code> ，用于图书分类筛选。
students	<code>created_at</code> 、 <code>full_name</code> 、 <code>class_name</code> ，用于学生借阅对象管理。
books	<code>created_at</code> 、 <code>title</code> 、 <code>author</code> 、 <code>location</code> 、 <code>cover_image_url</code> 、 <code>status</code> 、 <code>last_updated_by</code> 、 <code>total_quantity</code> 、 <code>available_quantity</code> 、 <code>category_id</code> 。
borrow_records	<code>created_at</code> 、 <code>book_id</code> 、 <code>student_id</code> 、 <code>profile_id</code> 、 <code>borrow_date</code> 、 <code>due_date</code> 、 <code>return_date</code> 、 <code>borrowed_by_user_id</code> 、 <code>quantity</code> 。

PocketBase 的 `record id` 是系统字段，不能创建自定义数字 `id` 字段。因此迁移时将 Supabase 的数字主键保存为 15 位补零 `record id`，例如源库 6 映射为 `000000000000006`。Flutter 端通过 `pb_mapper.dart` 将 PocketBase `id` 还原为业务模型中的整数 `id`。对于 `profiles`，完整 Supabase UUID 保存在 `source_id`，PocketBase `record id` 使用 UUID 去横线后的兼容短 `id`。

2.4.1 字段映射原则

字段迁移遵循三条原则。第一，原 Supabase 中已经存在的业务字段必须保留，不为了新数据库重新命名或重新设计语义；第二，PocketBase 的系统字段不与业务字段冲突，例如 `record id` 由 PocketBase 管理，原数字 `id` 通过可逆规则映射；第三，空值必须保留为空值，不能把空分类、空归还日期或空学生关系错误写成 `0` 或空字符串。

在数据类型上，`created_at`、`borrow_date`、`due_date` 和 `return_date` 统一按时间字段或可空时间字段处理；`total_quantity`、`available_quantity` 和 `quantity` 按数字处理；`category_id`、`student_id`、`profile_id` 等关系字段由于源库存在 `nullable` 情况，在 PocketBase 侧使用兼容类型保存，再由 Flutter `mapper` 转换为模型中的可空整数或字符串。这样的处理避免了迁移后“数据能导入但页面解释错误”的问题。

2.4.2 数据一致性约束

图书库存一致性是系统最重要的业务约束。对于每一本图书，应满足：

```
available_quantity = total_quantity - sum(unreturned borrow_records.quantity)
```

其中未归还记录指 `return_date` 为空的借阅记录。系统页面展示的“已借出”数量由 `total_quantity - available_quantity` 得到，个人中心和图书详情展示的借阅数量则来自 `borrow_records.quantity`。此前调试中出现“图书详情显示借出6本，但个人中心只看到3本”的疑问，本质上是图书维度统计全部未归还记录，而个人中心只统计当前登录用户的记录，两者口径不同。设计文档中明确统计口径后，可以避免把正常的多用户借阅误认为数据异常。

2.5 借阅业务流程

借阅与归还是系统中容易出现数据不一致的业务。系统设计时将库存校验、记录写入和库存扣减作为一个顺序业务链处理，并在失败时回滚库存变化。图2.3展示了借阅与归还流程。

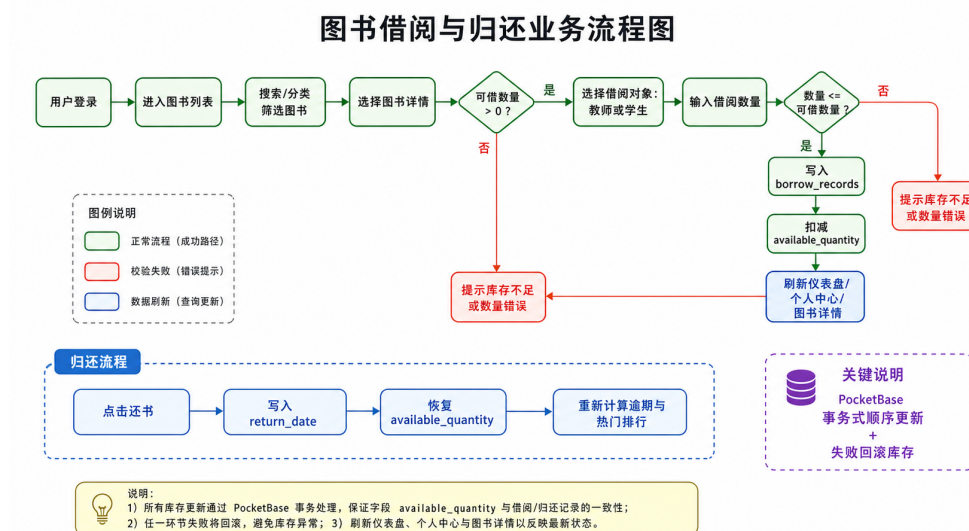


图 2.3 图书借阅与归还业务流程图

借阅流程首先读取图书当前可借数量，若输入数量大于可借库存则直接提示错误；校验通过后创建 `borrow_records`，再扣减 `books.available_quantity`。归还流程通过写入 `return_date` 标记记录已归还，并按借阅数量恢复可借库存。仪表盘、图书详情和个人中心均基于这些字段实时计算当前在借、逾期和历史记录。

2.6 接口调用时序

图2.4为登录、查询和借阅的端到端时序图。该图体现了鸿蒙 Flutter UI、Dart 服务层、PocketBase SDK、REST API 和 Docker volume 存储之间的调用关系。

登录阶段调用 `profiles.authWithPassword`，成功后将 `token` 与用户角色交给

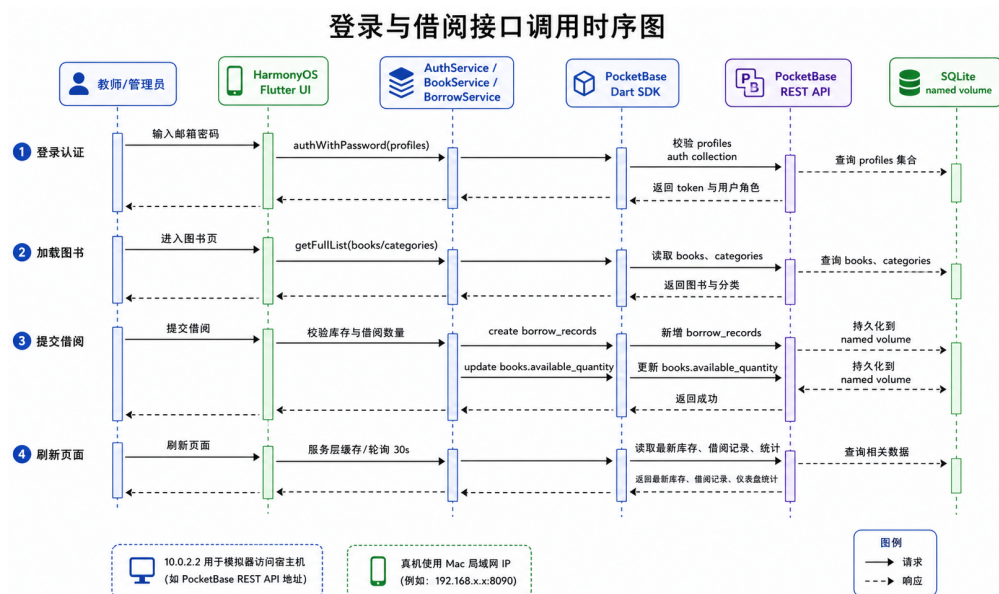


图 2.4 登录与借阅接口调用时序图

AuthService 管理。图书页进入时并行读取图书和分类数据；借阅提交时通过服务层完成库存校验、借阅记录创建和图书库存更新。为了避免模拟器中反复刷新导致的服务压力，列表类数据的轮询间隔统一收敛到 30 秒，并保留上一次成功结果作为短暂网络异常时的降级显示。

接口设计上，系统尽量避免在页面层拼接复杂查询。页面只表达用户意图，例如“加载图书列表”“加载某学生的借阅记录”“归还某条记录”；具体的 PocketBase collection 名称、排序字段、过滤条件和异常处理由服务层负责。这样做的好处是当 PocketBase 对非法字段排序返回 400 时，只需要修正服务层查询，不需要逐个修改页面组件。

2.7 数据库迁移与性能优化设计

本项目早期使用 Supabase，但在鸿蒙迁移过程中需要同时考虑平台 SDK 适配、本地验收可复现性和网络稳定性。PocketBase 的优势在于部署简单、提供标准 REST API、内置 auth collection 和静态文件服务，能够用 Docker 在本机快速启动，适合课程项目验收^[3-4,10]。

图2.5展示了数据库迁移与性能稳定性优化的整体思路。

迁移脚本 `tool/setup_pocketbase.dart` 从 Supabase 备份文件读取 `auth.users` 和 `public schema` 中的 5 个业务表，创建 PocketBase collections 后导入数据。迁移后的记录数量为：profiles 3 条、categories 4 条、students 4 条、books 6 条、borrow_records 8 条。系统没有为了新数据库重新设计字段，而是围绕 PocketBase

表 2.4 主要接口调用设计

服务方法	PocketBase 资源	说明
login	profiles/auth-with-password	使用邮箱密码登录,并读取角色字段。
getBooks	books/records	分页读取图书,按创建时间或标题排序。
getCategories	categories/records	读取分类,用于筛选按钮和图书编辑。
borrowBook	borrow_records/records、books/records	创建借阅记录并扣减库存。
returnBook	borrow_records/records、books/records	写入归还日期并恢复库存。
getDashboardStats	多个 collection	聚合图书、学生、借阅记录并生成统计值。

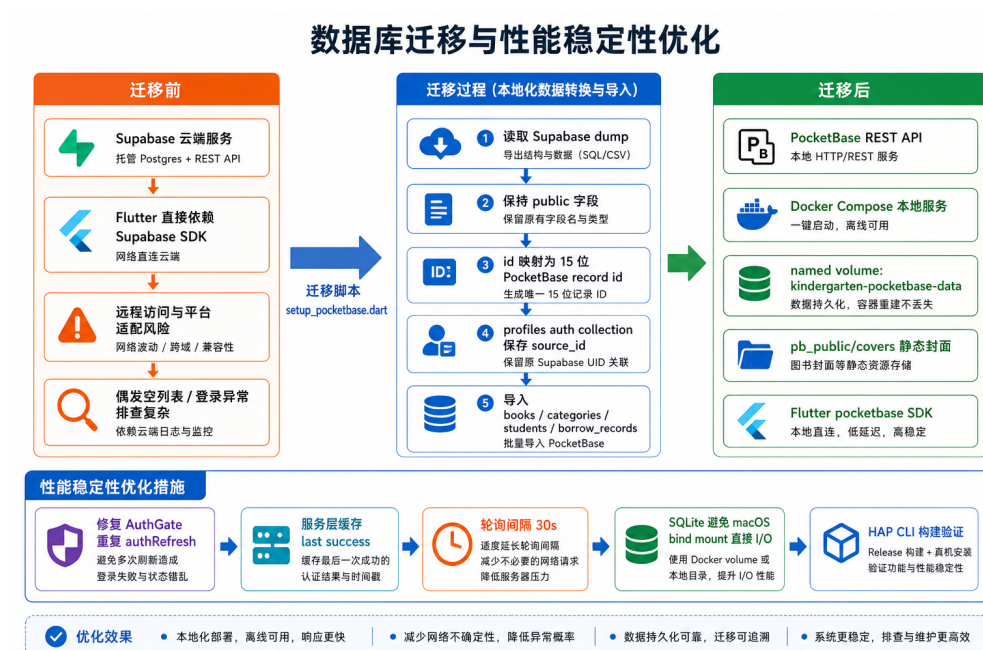


图 2.5 数据库迁移与性能稳定性优化图

的 `id`、`auth` 和 `nullable` 字段限制做最小适配。

性能优化主要包含四点：第一，将 `/pb_data` 放入 Docker named volume，避免 macOS Docker bind mount 直接承载 SQLite 高频读写；第二，封面图片放入 `pb_public/covers`，由 PocketBase 静态目录直接提供；第三，修复 AuthGate 重复触发 `authRefresh` 的问题，减少无意义认证请求；第四，服务层保留上一次成功数据，避免瞬时请求失败时把页面误渲染成“暂无图书”。该设计与 Docker 对 named volume 的推荐用途以及 SQLite WAL/文件锁机制有关^[9,11]。

迁移后的性能提升主要体现在三个方面。第一，服务启动路径缩短，验收时只需要启动本机 Docker 容器，不再依赖远程 Supabase 项目的网络和访问密钥；第二，数据库与图片资源位于同一个本地服务中，图书列表无需跨多个域名加载封面；第三，读写异常更容易定位，开发者可以直接查看 PocketBase 日志、collection 记录和 Docker volume 状态。对于课程项目而言，这种“可复现、可观察、可恢复”的工程属性比单纯追求云端能力更重要。

第3章 系统具体实现

3.1 鸿蒙工程适配实现

本项目采用 Flutter for OpenHarmony 3.22 生成鸿蒙工程。Flutter 本身采用 Dart 编写 UI 框架并通过自身渲染管线跨平台输出界面，适合在保留原前端的前提下迁移到新平台^[1]。生成命令如下：

```
/Users/achilles/development/flutter_flutter_3_22_ohos/bin/flutter \  
create --platforms ohos --project-name kindergarten_library --no-pub .
```

生成后的 ohos/ 工程可以由 DevEco Studio 打开，主要配置文件如下。OpenHarmony Stage 模型中，HAP 结构与 module.json5 配置共同决定模块类型、入口 Ability 和权限声明^[5-6,8]：

- ohos/build-profile.json5：配置 HarmonyOS 产品、签名信息、兼容 SDK 版本和目标 SDK 版本。
- ohos/entry/src/main/module.json5：声明入口模块、设备类型、EntryAbility 和网络访问权限。
- Index.ets：通过 @ohos/flutter_ohos 嵌入 FlutterPage，使原 Flutter 页面在鸿蒙 Stage 模型下启动。

该适配方案的关键优点是：页面层、模型层和服务层基本保持 Dart 实现，鸿蒙工程主要承担应用壳、权限、构建配置和 Flutter 容器职责。这比完全改写为 ArkTS 页面更适合本课程项目的时间范围，也更容易保持原有业务逻辑正确性。

3.1.1 鸿蒙工程目录说明

生成 ohos/ 后，项目不再只是一个普通 Flutter 工程，而是同时包含鸿蒙原生工程结构。顶层 build-profile.json5 描述产品、模块和签名信息；entry 模块是应用入口；hvmigorfile.ts 与 oh-package.json5 参与鸿蒙侧构建；oh_modules 和 .hvmigor 保存 OpenHarmony 依赖和构建缓存。DevEco Studio 能够打开该目录，是因为目录结构已经满足 OpenHarmony Stage 工程的基本要求。

入口文件 Index.ets 的职责相对清晰：它不重新实现登录页、图书页或仪表盘，而是创建 Flutter 容器并加载 Dart 侧应用。这一设计体现了 Flutter for OpenHarmony 的迁移价值。课程项目要求体现鸿蒙开发过程，因此本文不仅保留了 Flutter 前端，也在工程中配置了 HarmonyOS SDK、HAP 构建、网络权限、EntryAbility 和 DevEco

Studio 可识别的模块结构。

3.1.2 权限与网络配置

移动端访问 PocketBase 必须具备网络权限。系统在 `module.json5` 中声明 `ohos.permission.INTERNET`，否则即使 PocketBase 服务正常启动，鸿蒙端也无法发起 HTTP 请求。配置文件同时声明入口 Ability、设备类型和模块信息，使应用能在手机模拟器上安装和启动。

网络地址根据运行环境区分。桌面浏览器访问 PocketBase 使用 `localhost:8090`；鸿蒙模拟器中的 `127.0.0.1` 指向模拟器自身，因此必须使用 `10.0.2.2:8090` 访问 Mac 宿主机；真机验收时则需要把 Mac 和手机连接到同一局域网，再使用 Mac 的局域网 IP。该差异通过 `--dart-define` 暴露为配置项，避免在源码中反复修改地址。

3.2 后端连接与鉴权实现

Flutter 端使用 `lib/config/backend_config.dart` 管理 PocketBase 地址。默认值为 `http://10.0.2.2:8090`，用于鸿蒙模拟器访问 Mac 宿主机；真机环境可以通过 `POCKETBASE_URL` 编译参数切换为局域网地址。封面图片如果是相对路径，例如 `/covers/book_001.png`，会通过 `resolveFileUrl` 转换为完整 URL。

PocketBase 客户端初始化位于服务层的 `pocketbase_client.dart`。系统优先使用 `shared_preferences` 持久化 auth token；考虑到 OpenHarmony 插件适配可能不完整，当遇到 `MissingPluginException` 时回退为内存 auth store，保证应用至少能够在当前运行会话内完成登录和数据访问。该部分使用 PocketBase Dart SDK 与 `auth collection` 的邮箱密码认证能力^[7,12]。

`AuthService` 封装登录、注册、刷新当前用户和资料更新逻辑。登录时调用 `profiles auth collection` 的邮箱密码认证接口，成功后读取 `role` 决定用户权限。为了避免之前出现的重复刷新问题，应用入口 `AuthGate` 不再在每次 `build` 后无条件触发登录态更新，而是让 `auth` 状态流驱动页面切换。

3.2.1 服务层异常处理

服务层异常处理分为三类。第一类是连接异常，例如 PocketBase 未启动、模拟器地址错误或真机无法访问 Mac，此时页面显示明确的失败提示，并保留上一次成功数据。第二类是业务异常，例如库存不足、必填字段为空或归还已经完成的记录，此时服务层返回可读错误信息。第三类是后端查询异常，例如字段名写错、

排序字段不存在或过滤条件不兼容，此时服务层需要修正查询表达式，而不是让页面长期显示空态。

为了减少偶发空列表，**BookService**、**DashboardService** 和借阅记录相关服务采用“成功结果缓存”策略。每次请求成功后更新缓存；如果下一次请求失败且缓存存在，页面继续展示旧数据并提示刷新失败。这一策略并不会伪造数据，只是避免把网络瞬时失败解释为业务数据不存在。对于移动端应用，这比简单地在异常时返回空数组更符合用户预期。

3.2.2 角色路由实现

登录成功后，系统根据 **profiles.role** 判断当前用户是普通教师还是管理员。普通教师进入后可以使用图书查询、借阅、归还和个人中心；管理员可以进入分类、学生、全部借阅记录和逾期查询等管理页面。页面上的入口按钮和底部导航会根据角色展示不同功能，减少普通教师误进入管理功能的可能。

这种角色控制主要在前端完成，适合课程项目演示和验收。若系统进一步用于真实生产场景，还需要在 **PocketBase** 规则中增加 **collection** 级别的访问控制，例如限制普通教师只能更新自己的 **profiles**，只能读取必要的学生信息，不能删除分类或学生记录。本文在展望部分也将其列为后续改进方向。

3.3 仪表盘模块

仪表盘是管理员和教师了解图书馆当前状态的入口，如图3.1(a)所示。页面显示图书总数、学生总数、当前在借、逾期未还、本月借阅次数、热门图书和借阅之星。

DashboardService 会读取图书、学生和借阅记录，并在前端计算统计值。为了减少请求次数，当前实现会复用已经读取的借阅记录计算当前在借和逾期数量，而不是为每个统计卡片重新发起请求。热门图书根据借阅记录中的图书 **id** 和数量聚合得到，借阅之星根据借阅人信息聚合得到。

仪表盘的数据计算口径如下：图书总数来自 **books** 记录数量；学生总数来自 **students** 记录数量；当前在借来自 **return_date** 为空的借阅记录数量或数量合计；逾期未还来自未归还记录中 **due_date** 早于当前日期的记录；本月借阅次数按 **borrow_date** 所在月份过滤；热门图书按 **book_id** 聚合 **quantity** 后排序。

由于这些统计值来自多个 **collection**，如果每个卡片都单独请求后端，页面会产生过多请求并增加模拟器环境下的不稳定性。因此实现时将统计计算集中在一个服务方法内完成，页面一次加载后统一刷新。该方式也方便测试，因为可以通



图 3.1 仪表盘与图书总览页面

过固定的图书、学生和借阅记录推导出预期统计结果。

3.4 图书管理模块

图书管理页面是系统使用频率最高的页面，如图3.1(b)所示。用户可以通过搜索框按书名、作者或位置检索，也可以通过分类按钮筛选图书。每张图书卡片展示封面区域、书名、作者、库存和可借状态。

BookService 负责图书的增删改查。读取图书列表时会同时读取分类集合，并在本地为图书补充分类名称。若分类映射短暂失败，页面不会把全部图书隐藏，而是优先展示上一次成功加载的数据。这样的处理可以避免后端瞬时抖动时出现“暂无图书”的误判。

图书编辑页面如图3.2(a)所示。编辑内容包括标题、作者、位置、分类、库存、封面路径和状态等。系统在保存前对必填字段和库存数字进行校验，保存后刷新图书列表与详情页。

图书卡片在鸿蒙模拟器上曾出现过图片长期加载、按钮被遮挡和库存标签溢出等问题。修复时从三个方面处理：第一，封面区域设置固定高度，图片加载失败时显示占位而不是撑开卡片；第二，库存标签、可借状态和按钮采用紧凑布局，避免窄屏下横向越界；第三，搜索框和分类按钮放在独立滚动区域，分类较多时允

许横向滑动。这样可以保证图书页在不同屏幕宽度下仍保持稳定布局。

3.5 分类管理模块

分类管理页面如图3.2(b)所示，用于维护图书分类集合。分类数据来自 `categories collection`，图书表中的 `category_id` 与分类 `id` 对应。页面提供分类查看、添加、编辑和删除能力，图书列表顶部的分类筛选按钮也直接来自该集合。



(a) 编辑图书信息

(b) 图书分类管理

图 3.2 图书信息维护与分类管理页面

由于 `PocketBase` 的 `json` 字段可以保存 `nullable` 数字，迁移时 `books.category_id` 使用 `json` 类型，避免空分类被错误转换为 `0`。`Flutter` 模型读取时通过统一 `mapper` 将 `json`、`number` 或字符串形式转换为可空整数。

分类删除需要考虑图书引用关系。当前课程项目中分类数量较少，删除分类前主要通过界面提醒避免误操作；如果后续用于真实场景，应在服务层检查是否仍有图书引用该分类，或者把对应图书的 `category_id` 置空。这样可以防止分类删除后图书列表出现无法解析的分类名称。

3.6 借阅与归还模块

借阅模块支持两种对象：学生借阅和教师/管理员借阅。借阅时需要选择图书、借阅对象、借阅数量和归还期限。系统根据 `available_quantity` 判断是否允许借

阅，并通过 `quantity` 字段记录一次借阅的本数。

图3.3(a)展示了按图书查看借阅人员的页面。该页面用于回答“某本书目前由谁借阅、借了几本、是否逾期”等问题。

管理员可以查看全部借阅记录，如图3.3(b)所示。记录页面支持按图书名称、学生姓名或经办老师搜索，并提供全部、已归还、未归还、逾期等筛选条件。此前出现过 PocketBase 对非法排序或字段过滤返回 400 的问题，修复时将查询字段与排序字段严格限制在实际存在的 `collection` 字段上。

逾期未还页面如图3.3(c)所示。系统以 `return_date` 是否为空判断是否仍在借，以 `due_date` 与当前日期比较判断是否逾期。页面会显示逾期天数、图书、借阅人、班级或角色、借阅日期和应还日期。

借阅记录的显示需要同时解析图书、学生和用户资料。对于学生借阅，页面优先显示学生姓名和班级；对于教师借阅，页面显示教师姓名和角色。此前页面中出现过“null - 未分配班级”的情况，根因是教师借阅没有学生班级字段，而页面统一按学生展示方式拼接文本。修复后页面按 `student_id` 和 `profile_id` 区分借阅对象，避免把教师借阅误显示为学生班级缺失。

借阅数量也需要保持统一口径。一次借阅可以包含多本同名图书，因此 `quantity` 不应被忽略。图书详情中的“已借出”表示所有未归还记录的数量总和；个人中心的“我的借阅”只展示当前登录用户的记录；管理员记录页展示全局记录。三个页面统计对象不同，但都基于同一张 `borrow_records` 表计算。

3.7 学生管理模块

学生管理模块用于维护幼儿园学生信息，如图3.4(a)所示。学生记录包括姓名和班级，数据存储在 `students collection` 中。管理员可以新增、编辑或删除学生，借阅页面可选择学生作为借阅对象。

学生详情页如图3.4(b)所示，页面展示学生基础信息、当前借阅和历史借阅。对于学生借阅，`borrow_records.student_id` 存储学生 `id`；对于教师/管理员借阅，`profile_id` 存储用户 `id`。两者二选一，保持了原 Supabase 业务语义。

学生模块与图书模块的关系不是简单的主从关系。学生可以借阅多本图书，同一本图书也可以被不同学生或教师借阅，二者通过借阅记录关联。因此学生详情页并不把图书数据冗余存入学生记录，而是按 `student_id` 查询借阅记录，再根据 `book_id` 补充图书名称和作者。这样的设计能减少数据冗余，也能保证图书标题修改后历史记录页面仍显示最新图书信息。

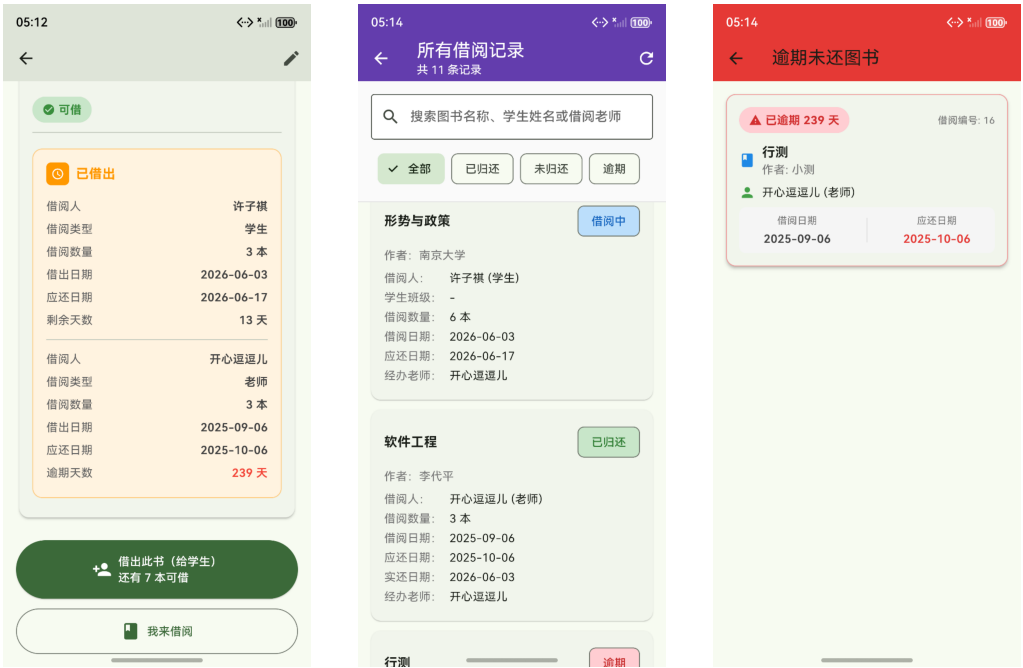


图 3.3 借阅记录、管理员记录与逾期查询页面



图 3.4 学生列表与学生借阅详情页面

3.8 图片加载实现

原 Supabase 数据中封面地址为远程 URL。迁移到 PocketBase 后，封面路径统一改为 `/covers/book_*.png`，图片文件放在 `pocketbase/pb_public/covers`。PocketBase 会将 `collection` 文件和静态资源作为后端资源暴露，因此前端只需要把相对路径拼接到 PocketBase base URL 即可^[3,12]。

在鸿蒙模拟器中，`127.0.0.1` 指向模拟器自身，不是 Mac 宿主机。因此图片加载和 API 请求都必须使用 `10.0.2.2` 或真机可访问的局域网 IP。此前图片一直显示加载圆环，根因就是 URL 指向或静态目录不可达，而不是 Flutter 图片组件本身无法渲染。

图片加载链路可以拆分为四个环节：数据库字段保存相对路径、PocketBase 静态目录存在对应文件、前端根据 base URL 拼接完整地址、图片组件发起网络请求并渲染。任一环节错误都会表现为封面空白或持续加载。最终方案选择相对路径而不是写死完整 URL，是为了同时兼容模拟器和真机；只要 base URL 切换正确，数据库中的封面路径无需再次迁移。

3.9 界面适配与交互实现

鸿蒙课程项目不仅要求功能可运行，也要求界面在模拟器和真机上具备基本可用性。本项目页面主要面向手机竖屏设计，因此采用底部导航、卡片列表、横向分类筛选和表单弹窗等移动端常见交互。仪表盘使用统计卡片快速展示概况，图书页使用双列卡片提高浏览效率，学生和借阅记录页面则更偏向列表式展示，便于查看文本信息。

界面适配中最容易出问题的是窄屏横向空间。借阅详情页中的“借阅数量”标签、还书按钮和日期文字曾出现溢出提示。修复时将长文本放入可伸缩区域，按钮设置最小宽度和固定高度，并避免在同一行同时放置过多信息。对于必须展示的数字标签，例如“×6”借阅数量，使用独立的小标签承载，不再让其挤压标题和日期。

交互反馈方面，系统在登录失败、库存不足、网络异常、保存成功和归还成功等关键节点提供提示。提示内容尽量面向业务含义，而不是直接暴露底层异常。例如连接失败时提示检查 PocketBase 服务和地址配置；字段错误时提示表单输入；后台返回 `400` 或 `500` 时在调试阶段显示完整信息，方便定位 `collection` 字段或服务状态问题。

第 4 章 测试与运行验证

4.1 测试环境

项目主要在 macOS + DevEco Studio 环境下完成开发和验证。测试环境如表4.1所示。

表 4.1 测试与构建环境

项目	配置
移动端 SDK	Flutter for OpenHarmony 3.22, Dart 3.4。
鸿蒙工具链	DevEco Studio 6.0.2, HarmonyOS SDK API 22, ohpm 6.0.1, hvigor。
后端服务	Docker Compose + PocketBase, 端口 8090。
数据库持久化	Docker named volume kindergarten-pocketbase-data。
调试设备	HarmonyOS 模拟器 nova 15 Pro

测试环境设计遵循“本地后端可控、移动端真实构建、虚拟机人工验收”的原则。后端通过 Docker 在 Mac 上运行，数据库和静态资源都由 PocketBase 提供；移动端先通过 CLI 构建 HAP，证明鸿蒙工程结构、SDK 配置和依赖解析正确；最终运行效果再交由鸿蒙模拟器或真机进行人工验收。这样的流程可以把构建问题、网络问题和界面问题分开排查。

在调试过程中，浏览器访问、Flutter 调试和鸿蒙模拟器访问使用的是不同网络视角。浏览器访问 localhost 表示 Mac 本机；Flutter 桌面或普通命令行也可以访问 localhost；鸿蒙模拟器则需要使用 10.0.2.2。因此测试时必须同时检查 PocketBase 是否启动、端口是否映射、移动端 base URL 是否正确、图片静态目录是否存在。

4.2 数据迁移验证

迁移脚本读取如下 Supabase 备份文件，导入结果如表4.2所示。

```
supabase_restore_work/db_cluster-16-09-2025@15-17-22.backup
```

迁移后系统登录账号如表4.3所示。三个应用用户密码统一为 12345678，便于课程验收时快速登录不同角色。

PocketBase 后台超级用户为 admin@example.com / admin123456，仅用于访问

表 4.2 迁移记录数量

来源数据	迁移数量
auth.users	3
public.profiles	3
public.categories	4
public.students	4
public.books	6
public.borrow_records	8

表 4.3 应用登录账号

邮箱	密码	姓名	性质
1424408591@qq.com	12345678	奕晨司	普通教师
2633638634@qq.com	12345678	小明	普通教师
903398655@qq.com	12345678	开心逗逗儿	管理员

PocketBase 管理后台，不作为应用内教师账号使用。

数据迁移验证不只检查记录数量，还要检查字段语义是否正确。图书记录需要确认标题、作者、位置、封面路径、总库存和可借库存一致；借阅记录需要确认借阅日期、应还日期、归还日期、借阅人和数量一致；用户记录需要确认邮箱、姓名和角色一致。尤其是 `return_date`、`student_id`、`profile_id` 这类可空字段，如果迁移时被错误填充，会直接影响逾期判断和个人中心显示。

表 4.4 迁移字段核验要点

Collection	核验要点
profiles	邮箱可登录，姓名显示正确， <code>role</code> 能区分教师和管理员。
books	标题、作者、位置、分类、封面路径、总数量和可借数量与源库一致。
borrow_records	未归还记录的 <code>return_date</code> 保持为空，数量字段参与库存计算。
students	学生姓名和班级能在学生详情、借阅选择和管理员记录中显示。
categories	分类名称能在图书筛选按钮和图书编辑页面中复用。

4.3 功能测试

系统围绕主要业务路径进行了功能测试，结果如表4.5所示。

表 4.5 核心功能测试结果

模块	测试内容	结果
登录鉴权	使用教师和管理员账号登录，校验角色与页面跳转。	通过
仪表盘	查看图书总数、学生总数、在借数量、逾期数量和热门图书。	通过
图书管理	搜索、分类筛选、新增、编辑、库存展示和封面加载。	通过
分类管理	添加、修改、删除分类，并在图书筛选中同步展示。	通过
借阅归还	教师/学生借阅、数量校验、还书、逾期计算和库存恢复。	通过
借阅记录	管理员查看全部记录、未归还记录和逾期记录。	通过
学生管理	学生列表、学生详情、班级信息和个人借阅历史。	通过
鸿蒙界面	修复图书卡片、借阅数量、按钮等小屏幕溢出问题。	通过

除了表4.5中的主流程测试，还对边界情况进行了检查。移动端业务系统常见的问题并不总是出现在“正常点击”路径上，而是出现在空数据、错误地址、重复刷新、数量边界和页面回退等场景中。表4.6列出了本项目重点验证的边界用例。

表 4.6 边界情况测试

类别	测试内容	结果
网络地址	将后端地址错误配置为 127.0.0.1，确认模拟器连接失败并给出提示。	通过
库存边界	借阅数量大于可借库存，系统阻止提交。	通过
空分类	图书分类为空或分类读取失败时，图书仍能显示基础信息。	通过
图片失败	封面文件不存在或地址不可达时，页面显示占位，不影响列表滚动。	通过
重复刷新	多次切换底部导航和返回页面，列表不会频繁变成空态。	通过
角色权限	教师账号不展示管理员查询入口，管理员账号可进入管理页面。	通过

4.4 界面适配测试

界面测试重点关注手机竖屏下的元素溢出、点击区域和滚动体验。由于鸿蒙模拟器的状态栏、导航栏和侧边控制条会占用视觉空间，页面实际可用宽度比桌面预览更紧张。图书卡片、个人中心借阅卡片和管理员记录卡片都进行了窄屏检查。

主要调整包括：将并列文本改为可换行布局；将还书按钮和数量标签设置为固定尺寸；将长标题、日期和班级信息放入弹性区域；将分类筛选放入横向滚动

容器；对底部导航预留安全区域。调整后，页面在模拟器中不再出现 Flutter 的红色 overflow 提示，关键按钮也不会被数量标签或日期文本遮挡。

4.5 后端运行验证

PocketBase 服务验证包含三个层次。第一，Docker 层确认容器正在运行并暴露 8090 端口；第二，PocketBase 层确认管理后台可以打开、collection 存在且记录数量正确；第三，移动端层确认模拟器可以通过 10.0.2.2:8090 登录并读取图书。三层都通过后，才能说明后端不仅“启动了”，而且“能被鸿蒙端使用”。

常用检查命令如下：

```
docker compose ps
curl http://127.0.0.1:8090/api/health
curl http://127.0.0.1:8090/covers/book_001.png
```

其中图片检查用于验证静态资源目录是否正确挂载。若 API 正常但图片不显示，通常应优先检查 pb_public/covers 是否存在文件、数据库中的 cover_image_url 是否为相对路径，以及前端是否把相对路径拼接到了正确的 base URL。

4.6 构建验证

本项目已通过 Flutter 分析、单元测试和 HarmonyOS HAP 构建验证。常用命令如下：

```
flutter analyze --no-fatal-infos
flutter test --no-pub
```

鸿蒙构建前需要显式设置 DevEco 工具链环境变量，并使用 Flutter for OpenHarmony SDK 执行 HAP 构建。HAP 是 OpenHarmony 应用模块的构建产物，包含代码、资源、三方库和应用配置文件^[8]：

```
export HOS_SDK_HOME=/Applications/DevEco-Studio.app/Contents/sdk
export DEVECO_SDK_HOME=/Applications/DevEco-Studio.app/Contents/sdk
export OHOS_SDK_HOME=/Applications/DevEco-Studio.app/Contents/sdk/default/openharmony
export OHOS_BASE_SDK_HOME=/Applications/DevEco-Studio.app/Contents/sdk/default/openharmony
export TOOL_HOME=/Applications/DevEco-Studio.app/Contents
export JAVA_HOME=/Applications/DevEco-Studio.app/Contents/jbr/Contents/Home
export PATH=/Users/achilles/development/flutter_flutter_3_22_ohos/bin:$TOOL_HOME/tools/ohpm/
↪ bin:$TOOL_HOME/tools/hvigor/bin:$TOOL_HOME/tools/node/bin:$JAVA_HOME/bin:$PATH

/Users/achilles/development/flutter_flutter_3_22_ohos/bin/flutter \
  build hap --debug --no-codesign --target-platform ohos-arm64
```

构建产物位于：

```
ohos/entry/build/default/outputs/default/entry-default-unsigned.hap
```

HAP 构建验证的意义在于确认项目已经从普通 Flutter 应用转化为可被 HarmonyOS 工具链识别的应用模块。若 ohos/ 工程缺失、module.json5 配置错误、SDK 路径错误或 ohpm 依赖未解析，构建阶段就会失败。通过 CLI 构建后，即使暂不使用虚拟机自动安装，也可以说明鸿蒙工程适配已经具备可验收基础。

构建过程中还需要注意签名问题。课程调试阶段可以使用 `--no-codesign` 生成未签名 HAP，用于证明编译链路可通；正式真机安装时需要按照 DevEco Studio 的签名配置生成可安装包。本文以课程开发验证为目标，因此采用 `debug + no-codesign` 的方式完成命令行验证。

4.7 稳定性问题分析

项目调试过程中出现过两类典型问题。第一类是模拟器无法连接后端，表现为 `Connection refused` 或状态码为 `0`。原因是模拟器内的 `127.0.0.1` 指向设备自身，而 PocketBase 实际运行在 Mac 宿主机。修复方式是默认使用 `10.0.2.2:8090`，真机再切换为 Mac 局域网 IP。

第二类是偶发空列表或统计数据异常。排查后发现不是业务数据丢失，而是重复鉴权刷新、过密轮询、Docker bind mount 上的 SQLite I/O 抖动和瞬时请求失败叠加导致。SQLite WAL 会在数据库文件旁维护额外日志文件并涉及检查点写回，因此将数据库文件放在 Docker 管理的 volume 中更符合本项目的稳定性目标^[9,11]。修复后系统采取了以下策略：

- AuthGate 不再在每次 build 后重复触发 `authRefresh`。
- 列表轮询间隔收敛到 30 秒，减少高频请求。
- BookService 和 DashboardService 保留上一次成功数据，瞬时失败时不立即显示空态。
- PocketBase 的 `/pb_data` 使用 Docker named volume，不再直接放在 macOS bind mount 上。

这些优化使系统更适合鸿蒙模拟器和真机环境下的连续操作，也让课程验收时的后端部署更容易复现。

4.8 问题定位复盘

项目调试过程中的问题具有明显的跨平台迁移特征。白屏问题通常不是页面没有写，而是入口 **Ability**、**Flutter** 容器或资源加载链路未完全启动；登录连接失败不是账号错误，而是模拟器访问宿主机地址错误；图片不显示不是图片组件失效，而是静态路径和 **base URL** 不匹配；偶发空列表也不是数据库数据丢失，而是请求失败后页面错误地使用空数组覆盖了原有数据。

通过这些问题可以总结出一条有效的定位顺序：先看后端是否启动，再看地址是否能从当前设备访问，再看接口返回字段是否符合模型，再看页面是否正确处理加载态和错误态。该顺序比直接修改 **UI** 更可靠，也符合移动端前后端分离系统的调试规律。

对于数据库稳定性问题，最终方案没有简单地增加重试次数，而是同时减少不必要请求、调整 **Docker volume**、修正服务层缓存和明确查询字段。这样处理后，系统在模拟器连续切换页面时更稳定，也能避免“偶发查不到数据”影响人工验收。

第 5 章 总结与展望

5.1 工作总结

本文完成了一套面向幼儿园图书室场景的鸿蒙移动端图书管理系统。系统保留 Flutter 前端主体，通过 Flutter for OpenHarmony 生成 ohos/ 工程，并在 ArkTS EntryAbility 中嵌入 Flutter 页面，实现了从原 Flutter 应用到 HarmonyOS 工程的平滑迁移。应用端已经覆盖登录、图书管理、分类管理、学生管理、借阅归还、逾期查询、管理员借阅记录和仪表盘统计等核心功能。

后端方面，系统将 Supabase 迁移为 PocketBase + Docker 的本地部署方案。迁移严格遵循原数据库字段和数据内容，围绕 PocketBase 的系统 id、auth collection 和 nullable 字段限制做最小适配；同时通过 Docker named volume 保存 /pb_data，通过 pb_public/covers 提供封面图片静态资源。相比依赖远程 Supabase 服务，本地 PocketBase 后端在课程验收中更易启动、更易复现，也更便于定位网络与数据问题^[3,9-10]。

在工程稳定性方面，系统修复了模拟器访问宿主机地址、重复鉴权刷新、偶发空列表、图片路径解析和小屏幕界面溢出等问题，并通过 Flutter 分析、单元测试和 HarmonyOS HAP 构建验证，形成了完整的开发、迁移、部署和构建闭环。

从课程目标看，本项目覆盖了移动应用开发中的多个关键环节：界面设计、状态管理、网络请求、数据持久化、权限配置、跨平台构建和部署验证。尤其是在鸿蒙适配部分，项目没有停留在 Android 或 Web 运行效果，而是生成了真实 ohos/ 工程，并围绕 HarmonyOS SDK、DevEco Studio、HAP 构建和模拟器网络进行了实际调试。

从软件工程角度看，项目还体现了需求拆分、模块化设计、数据迁移、测试验证和问题复盘。数据库从 Supabase 切换到 PocketBase 并不是简单替换 URL，而是涉及字段映射、认证模型、图片资源、Docker volume、移动端配置和服务层容错的一组系统性修改。最终系统能够在本地后端和鸿蒙端之间形成稳定闭环，说明改造方案具有可实施性。

5.2 不足之处

受课程项目时间限制，系统仍存在一些可以继续优化的地方：

- 当前 OpenHarmony 下 shared_preferences 插件可能存在适配差异，因此客

户端提供了内存 `auth store` 回退方案；后续可接入更稳定的鸿蒙本地持久化插件。

- 目前 **PocketBase** 运行在本机 **Docker** 中，适合课程验收和局域网测试；若用于真实园所，需要进一步设计备份、权限、**HTTPS** 和运维策略。
- 封面图片当前由静态目录提供，已经解决显示问题，但尚未实现完善的移动端上传、压缩和图片管理流程。
- 借阅记录已有数量字段和库存联动，但仍可进一步加强并发借阅场景下的事务一致性。

此外，当前测试主要依赖开发者手动操作和命令行构建验证，自动化端到端测试仍不充分。由于鸿蒙模拟器的自动化测试环境搭建成本较高，本文没有实现完整的 UI 自动化脚本。后续如果项目继续演进，可以将服务层逻辑、数据 `mapper`、库存计算和日期判断纳入更系统的单元测试，降低后续修改引入回归的风险。

5.3 未来展望

后续可以从以下方向继续扩展系统：

1. 接入条形码或 **ISBN** 扫描，减少教师手工录入图书信息的成本。
2. 增加借阅到期提醒、班级借阅报表和月度统计导出功能。
3. 为管理员提供 **PocketBase** 数据备份、恢复和健康检查页面。
4. 针对 **HarmonyOS** 真机完善本地存储、图片选择、权限申请和网络错误提示。
5. 将教师、管理员和园所负责人权限进一步细分，支持更细粒度的数据访问控制。

在部署方面，未来可以把 **PocketBase** 从本机 **Docker** 迁移到园所内网服务器或轻量云服务器，并为移动端配置 **HTTPS** 域名。这样教师真机不再依赖开发者 **Mac** 的局域网地址，数据也可以由统一服务器集中保存。在权限方面，可以进一步利用 **PocketBase collection rules**，把教师、管理员和园所负责人区分为不同访问级别，使前端展示权限和后端数据权限保持一致。

在鸿蒙能力方面，后续可以继续接入系统相册、扫码、消息通知和本地安全存储等能力。例如通过扫码快速录入图书编号，通过本地安全存储保存登录 `token`，通过通知提醒即将逾期的图书。这些能力会进一步体现 **HarmonyOS** 原生生态价值，但也需要逐项验证对应插件或 **ArkTS** 原生能力的可用性。

总体来看，本项目不仅完成了一个可运行的幼儿园图书管理应用，也体现了鸿蒙移动端适配、数据库迁移、本地化部署、业务数据一致性和软件工程文档化等课程要求。

参考文献

- [1] Flutter Team. Flutter architectural overview[EB/OL]. 2026. <https://docs.flutter.dev/resources/architectural-overview>.
- [2] Supabase. Use supabase with flutter[EB/OL]. 2026. <https://supabase.com/docs/guides/getting-started/quickstarts/flutter>.
- [3] PocketBase. Introduction - collections[EB/OL]. 2026. <https://pocketbase.io/docs/collections/>.
- [4] Docker Inc. Docker compose documentation[EB/OL]. 2026. <https://docs.docker.com/compose/>.
- [5] OpenHarmony Project. Application package structure in stage model[EB/OL]. 2026. <https://gitee.com/openharmony/docs/blob/master/en/application-dev/quick-start/application-package-structure-stage.md>.
- [6] OpenHarmony Project. Module configuration file[EB/OL]. 2026. <https://gitee.com/openharmony/docs/blob/master/zh-cn/application-dev/quick-start/module-configuration-file.md>.
- [7] PocketBase. Introduction - authentication[EB/OL]. 2026. <https://pocketbase.io/docs/authentication/>.
- [8] OpenHarmony Project. Hap package[EB/OL]. 2026. <https://gitee.com/openharmony/docs/blob/master/en/application-dev/quick-start/hap-package.md>.
- [9] Docker Inc. Define and manage volumes in docker compose[EB/OL]. 2026. <https://docs.docker.com/reference/compose-file/volumes/>.
- [10] Supabase. Supabase auth[EB/OL]. 2026. <https://supabase.com/docs/guides/auth>.
- [11] SQLite. Write-ahead logging[EB/OL]. 2026. <https://www.sqlite.org/wal.html>.
- [12] PocketBase. Pocketbase dart sdk[EB/OL]. 2026. <https://github.com/pocketbase/dart-sdk>.

附录 A 附录：核心命令与目录说明

A.1 PocketBase 启动

```
docker compose up -d pocketbase
```

PocketBase 后台地址为 `http://localhost:8090/_/`。应用端在鸿蒙模拟器中默认通过 `http://10.0.2.2:8090` 访问宿主机服务。

如果需要查看服务状态，可以使用：

```
docker compose ps
docker compose logs -f pocketbase
```

如果需要停止服务，可以使用：

```
docker compose down
```

注意不要随意删除 Docker volume。系统数据库保存在 `kindergarten-pocketbase-data` 中，如果删除 volume，需要重新执行迁移脚本导入数据。

A.2 数据迁移

```
flutter pub get
dart run tool/setup_pocketbase.dart
```

迁移完成后应检查 PocketBase 后台中的 collection 和记录数量。应用账号密码统一设置为 12345678，便于课程验收切换不同用户角色。若迁移脚本重新执行，应确认旧 collection 是否会被覆盖，避免重复数据影响测试结果。

A.3 模拟器与真机地址

模拟器和真机访问后端的地址不同，常见配置如下：

真机调试时，可以通过如下命令指定后端地址：

```
flutter run -d <device-id> \
  --dart-define=POCKETBASE_URL=http://192.168.x.x:8090
```

表 A.1 后端地址配置

运行环境	PocketBase 地址
Mac 浏览器	http://127.0.0.1:8090 或 http://localhost:8090
鸿蒙模拟器	http://10.0.2.2:8090
鸿蒙真机	http://Mac 局域网 IP:8090

A.4 HarmonyOS HAP 构建

```
/Users/achilles/development/flutter_flutter_3_22_ohos/bin/flutter \
build hap --debug --no-codesign --target-platform ohos-arm64
```

构建前如果 DevEco 工具链不在环境变量中，需要先设置 HOS_SDK_HOME、OHOS_SDK_HOME、TOOL_HOME 和 JAVA_HOME。这些变量用于让 Flutter for OpenHarmony 找到 HarmonyOS SDK、ohpm、hvmor 和 JBR。

A.5 常见问题处理

表 A.2 常见问题与处理方式

现象	处理方式
登录提示连接失败	确认 PocketBase 已启动；模拟器使用 10.0.2.2，真机使用 Mac 局域网 IP。
图书列表偶发为空	检查后端日志和网络地址；服务层已保留上次成功数据，避免瞬时失败覆盖页面。
图片一直加载	检查 pb_public/covers 是否存在图片，以及数据库路径是否为 /covers/...
HAP 构建失败	检查 Flutter for OpenHarmony SDK、DevEco SDK、ohpm 和 hvmor 路径。
页面出现溢出	检查卡片宽度、按钮宽度、长文本换行和底部安全区域。

A.6 主要源码目录

- lib/screens: Flutter 页面，包括登录、图书、学生、借阅、仪表盘和个人中心。
- lib/services: 业务服务层，封装 PocketBase 数据访问和业务校验。
- lib/models: 图书、学生、分类、借阅记录 and 用户资料模型。

- **ohos**: Flutter for OpenHarmony 生成的 HarmonyOS 工程。
- **pocketbase**: PocketBase 静态资源、hooks 和本地维护目录。

A.7 验收建议

人工验收时建议按以下顺序操作：首先启动 **Docker PocketBase** 服务并确认后台可打开；其次使用管理员账号登录，查看仪表盘、图书、学生和全部借阅记录；然后使用普通教师账号登录，验证图书查询、借阅和个人中心；最后切换到图书详情和逾期页面，检查库存、数量和逾期天数是否一致。

如果验收设备是真机，需要保证手机和 **Mac** 在同一网络下，并且 **Mac** 防火墙没有阻止 **8090** 端口访问。真机无法访问时，可以先在同一 **Wi-Fi** 下用手机浏览器打开 `http://Mac 局域网 IP:8090/api/health` 进行排查。